

DATABASE SYSTEMS BOOTCAMP

Practice Assignment

Transactions • Joins • Indexing • Query Optimization

Complete every task using DB Fiddle (or any MySQL environment of your choice). Read each instruction carefully before writing your query — some tasks build on the schema you create in Part 1.

Before you start: you only need a browser and DB Fiddle (db-fiddle.com, set to MySQL). No installation or account is required.

Part 1 — Build Your Own Practice Schema

Before attempting the tasks below, design and create your own small database. Do not copy a schema from anywhere — write your own CREATE TABLE statements based on the description below, then insert your own sample data using INSERT statements.

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

Design tip: when filling in your orders table, deliberately leave a few customers with zero orders. You'll need that gap later for the Joins task to mean something.

Column tips: customer_id and restaurant_id should be whole numbers referencing the id of a row in another table. total can be a decimal (e.g. price in rupees). status can be a short text value like 'pending', 'completed', or 'cancelled'. created_at should be a date or datetime value.

Schema SQL

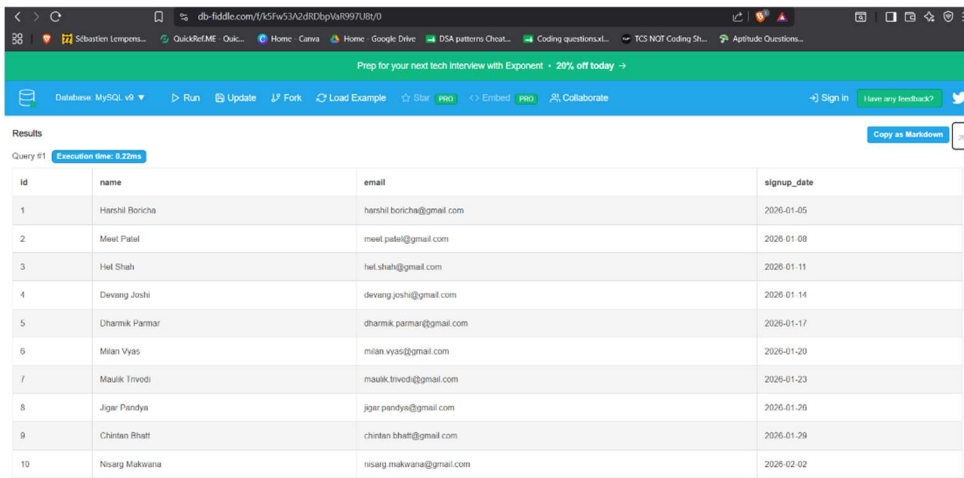
```
1 CREATE DATABASE FoodDelivery;
2 USE FoodDelivery;
3
4 CREATE TABLE customers (
5     id INT PRIMARY KEY AUTO_INCREMENT,
6     name VARCHAR(100) NOT NULL,
7     email VARCHAR(100) UNIQUE,
8     signup_date DATE
9 );
10
11 CREATE TABLE restaurants (
12     id INT PRIMARY KEY AUTO_INCREMENT,
13     name VARCHAR(100),
14     cuisine VARCHAR(50)
15 );
16
17 CREATE TABLE orders (
18     id INT PRIMARY KEY AUTO_INCREMENT,
19     customer_id INT,
20     restaurant_id INT,
21     total DECIMAL(10,2),
22     status VARCHAR(20),
23     created_at DATETIME,
24
25     FOREIGN KEY (customer_id) REFERENCES customers(id),
26     FOREIGN KEY (restaurant_id) REFERENCES restaurants(id)
27 );
28
29 CREATE TABLE accounts (
30     id INT PRIMARY KEY AUTO_INCREMENT,
31     name VARCHAR(50),
32     balance DECIMAL(10,2)
33 );
```

```
60 INSERT INTO orders (id, customer_id, restaurant_id, total, status, created_at) VALUES
70 (1,1,2,249.00,'completed','2026-07-02 12:15:00'),
71 (2,1,6,420.50,'pending','2026-07-08 19:20:00'),
72 (3,1,1,315.00,'completed','2026-07-10 13:10:00'),
73 (4,1,8,540.75,'cancelled','2026-07-15 20:30:00'),
74 (5,1,5,389.00,'completed','2026-07-20 18:45:00'),
75 (6,1,9,275.50,'pending','2026-08-02 14:20:00'),
76
77 (7,2,3,499.00,'completed','2026-07-03 13:00:00'),
78 (8,2,7,335.00,'pending','2026-07-08 20:10:00'),
79 (9,2,4,290.00,'completed','2026-07-14 09:45:00'),
80 (10,2,10,780.00,'cancelled','2026-07-25 19:15:00'),
81 (11,2,2,615.25,'completed','2026-08-03 18:00:00'),
82
83 (12,3,1,260.00,'pending','2026-07-04 11:00:00'),
84 (13,3,6,455.00,'completed','2026-07-12 18:30:00'),
85 (14,3,8,375.00,'completed','2026-07-24 15:45:00'),
86 (15,3,5,640.00,'cancelled','2026-08-05 20:00:00'),
87
88 (16,4,9,520.00,'completed','2026-07-06 12:00:00'),
89 (17,4,2,310.00,'pending','2026-07-16 18:20:00'),
90 (18,4,4,690.00,'completed','2026-07-28 20:10:00'),
91 (19,4,7,445.00,'cancelled','2026-08-06 13:25:00'),
92
93 (20,5,3,350.00,'completed','2026-07-07 11:25:00'),
94 (21,5,6,510.00,'pending','2026-07-18 19:40:00'),
95 (22,5,10,845.00,'completed','2026-07-30 20:25:00'),
96 (23,5,1,295.00,'cancelled','2026-08-07 14:50:00'),
97
98 (24,6,5,430.00,'completed','2026-07-09 13:30:00'),
99 (25,6,8,200.00,'pending','2026-07-22 17:40:00'),
100 (26,6,2,615.00,'completed','2026-08-08 19:10:00'),
101
102 (27,7,4,375.00,'completed','2026-07-11 10:20:00'),
103 (28,7,6,520.00,'cancelled','2026-07-26 18:50:00'),
104 (29,7,9,690.00,'completed','2026-08-09 20:15:00'),
105
```

```
INSERT INTO customers (id, name, email, signup_date) VALUES
(1,'Harshil Boricha','harshil.boricha@gmail.com','2026-01-05'),
(2,'Meet Patel','meet.patel@gmail.com','2026-01-08'),
(3,'Het Shah','het.shah@gmail.com','2026-01-11'),
(4,'Devang Joshi','devang.joshi@gmail.com','2026-01-14'),
(5,'Dharmik Parmar','dharmik.parmar@gmail.com','2026-01-17'),
(6,'Milan Vyas','milan.vyas@gmail.com','2026-01-20'),
(7,'Maulik Trivedi','maulik.trivedi@gmail.com','2026-01-23'),
(8,'Jigar Pandya','jigar.pandya@gmail.com','2026-01-26'),
(9,'Chintan Bhatt','chintan.bhatt@gmail.com','2026-01-29'),
(10,'Nisarg Makwana','nisarg.makwana@gmail.com','2026-02-02'),
(11,'Darshan Chauhan','darshan.chauhan@gmail.com','2026-02-05'),
(12,'Vatsal Dave','vatsal.dave@gmail.com','2026-02-08'),
(13,'Parth Thakkar','parth.thakkar@gmail.com','2026-02-11'),
(14,'Kishan Rabari','kishan.rabari@gmail.com','2026-02-14'),
(15,'Tushar Rathod','tushar.rathod@gmail.com','2026-02-17'),
(16,'Yuvraj Barot','yuvraj.barot@gmail.com','2026-02-20'),
(17,'Om Purohit','om.purohit@gmail.com','2026-02-23'),
(18,'Krunal Mistry','krunal.mistry@gmail.com','2026-02-26'),
(19,'Jenil Patel','jenil.patel@gmail.com','2026-03-01'),
(20,'Moksh Gandhi','moksh.gandhi@gmail.com','2026-03-04');
```

```
INSERT INTO restaurants (id, name, cuisine) VALUES
(1,'Gordhan Thal','Gujarati'),
(2,'Barbeque Nation','Barbecue'),
(3,'Tinello','Italian'),
(4,'Sankalp Restaurant','South Indian'),
(5,'Chinese Wok','Chinese'),
(6,'La Pino'z Pizza','Pizza'),
(7,'Jassi De Paratha','Punjabi'),
(8,'Makeba - The Lounge Cafe','Cafe'),
(9,'Taco Bell','Mexican'),
(10,'Sushi By Bou','Japanese');
```

```
122
123 INSERT INTO accounts (id, name, balance) VALUES
124 (1,'Alice',20000.00),
125 (2,'Bob',7000.00);
```



id	name	email	signup_date
1	Harshil Boricha	harshil.boricha@gmail.com	2026-01-05
2	Meet Patel	meet.patel@gmail.com	2026-01-08
3	Het Shah	het.shah@gmail.com	2026-01-11
4	Devang Joshi	devang.joshi@gmail.com	2026-01-14
5	Dharmik Parmar	dharmik.parmar@gmail.com	2026-01-17
6	Milan Vyas	milan.vyas@gmail.com	2026-01-20
7	Maulik Trivedi	maulik.trivedi@gmail.com	2026-01-23
8	Jigar Pandya	jigar.pandya@gmail.com	2026-01-26
9	Chintan Bhatt	chintan.bhatt@gmail.com	2026-01-29
10	Nisarg Makwana	nisarg.makwana@gmail.com	2026-02-02

Part 2 — Tasks

TASK 1 Transactions (ACID)

Using your accounts table:

- Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

```
Database: MySQL v9 ▼ ▶ Run
```

Query SQL ●

```
1 SELECT * FROM accounts;
2
3 START TRANSACTION;
4
5 UPDATE accounts
6 SET balance = balance - 200
7 WHERE name = 'Alice';
8
9 UPDATE accounts
10 SET balance = balance + 200
11 WHERE name = 'Bob';
12
13 ROLLBACK;
14
15 SELECT * FROM accounts;
```

Prep for your next tech interview with Exponent - 20% off today →

Database: MySQL v9 ▼ ▶ Run Update Fork Load Example Star PRO <> Embed PRO Collaborate Sign In Have any feedback? Twitter

Results [Copy as Markdown](#)

Query #1 **Execution time: 0.17ms**

id	name	balance
1	Alice	20000.0
2	Bob	7000.0

Query #2 **Execution time: 0.06ms**
There are no results to be displayed.

Query #3 **Execution time: 0.26ms**
There are no results to be displayed.

Query #4 **Execution time: 0.13ms**
There are no results to be displayed.

Query #5 **Execution time: 0.14ms**
There are no results to be displayed.

Query #6 **Execution time: 0.12ms**

id	name	balance
1	Alice	20000.0
2	Bob	7000.0

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

INPUT:

Query SQL •

```
1 START TRANSACTION;
2
3 UPDATE accounts
4 SET balance = balance - 200
5 WHERE id = 1;
6
7 UPDATE accounts
8 SET balance = balance + 200
9 WHERE id = 2;
10
11 COMMIT;
12
13 select * from accounts;
```

Query #5 Execution time: 0.15ms

id	name	balance
1	Alice	19800.0
2	Bob	7200.0

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

The **Atomicity** property of ACID guarantees that money is never lost during a transfer. It ensures that both operations (deducting money from one account and adding it to another) are completed together as a single transaction. If any part of the transaction fails, the entire transaction is rolled back, so the account balances remain unchanged.

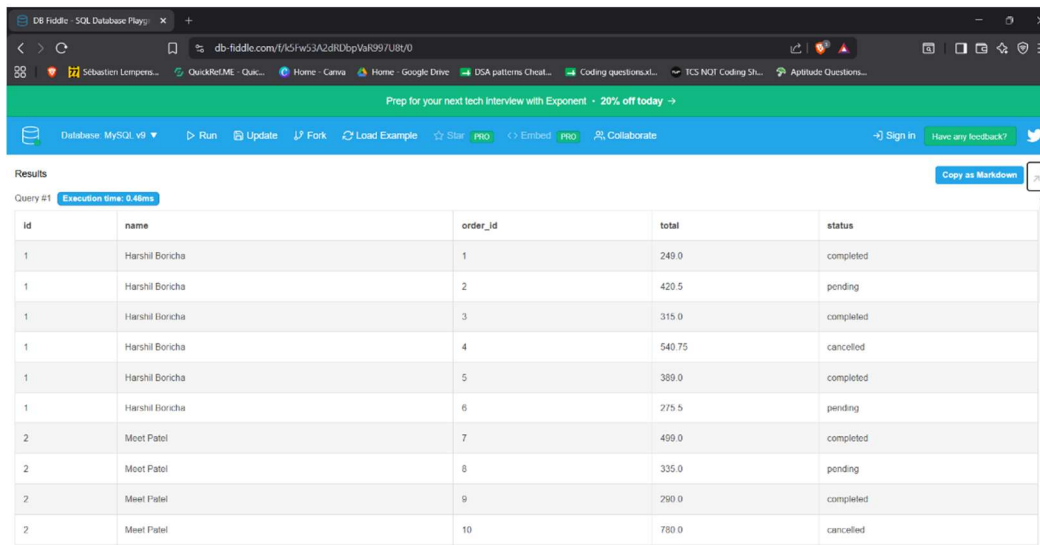
TASK 2 Joins

Using your customers and orders tables:

- a) Write an INNER JOIN that lists every customer who has placed at least one order.

Query SQL

```
1 SELECT
2     c.id,
3     c.name,
4     o.id AS order_id,
5     o.total,
6     o.status
7 FROM customers c
8 INNER JOIN orders o
9 ON c.id = o.customer_id;
```



The screenshot shows a web browser window with the URL `db-fiddle.com/1/Gfw3A2dRD0pVaR99/U8t/0`. The page displays the SQL query and its results. The results table has 5 columns: `id`, `name`, `order_id`, `total`, and `status`. The data is as follows:

id	name	order_id	total	status
1	Harshil Boricha	1	249.0	completed
1	Harshil Boricha	2	420.5	pending
1	Harshil Boricha	3	315.0	completed
1	Harshil Boricha	4	540.75	cancelled
1	Harshil Boricha	5	380.0	completed
1	Harshil Boricha	6	275.5	pending
2	Meet Patel	7	499.0	completed
2	Meet Patel	8	335.0	pending
2	Meet Patel	9	290.0	completed
2	Meet Patel	10	780.0	cancelled

- b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

Query SQL

```

1 SELECT
2     c.id,
3     c.name,
4     o.id AS order_id,
5     o.total
6 FROM customers c
7 LEFT JOIN orders o
8 ON c.id = o.customer_id;
9
10 SELECT
11     c.id,
12     c.name
13 FROM customers c
14 LEFT JOIN orders o
15 ON c.id = o.customer_id
16 WHERE o.id IS NULL;

```

Results

Query #1 Execution time: 9.4ms

Copy as Markdown

id	name	order_id	total
1	Harshil Boricha	1	249.0
1	Harshil Boricha	2	420.5
1	Harshil Boricha	3	315.0
1	Harshil Boricha	4	540.75
1	Harshil Boricha	5	389.0
1	Harshil Boricha	6	275.5
2	Meet Patel	7	499.0
2	Meet Patel	8	335.0
2	Meet Patel	9	290.0
2	Meet Patel	10	780.0
2	Meet Patel	11	615.25

DB Fiddle — Crafted with ♥ by StatuS200 in the United Kingdom.

Terms of Use • Privacy / Cookie Policy • StatuS200 Ltd © 20

Query #2 Execution time: 0.19ms

id	name
14	Kishan Rabari
15	Tushar Rathod
16	Yuvraj Barot
17	Om Purohit
18	Krunal Mistry
19	Jenil Patel
20	Moksh Gandhi

- c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

The marketing team would prefer the result of query **(b)** because it returns only the required columns, making the data easier to read, faster to retrieve, and more efficient for analysis.

TASK 3 Indexing

Using your orders table:

- a) Pick a column that is not currently indexed (for example status or created_at).

I selected the **status** column because it is frequently used to filter records, and indexing it can improve query performance..

- b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status='completed';
```

Results

Copy as Markdown

Query #1 Execution time: 0.36ms

EXPLAIN

-> Filter: (orders.'status' = 'completed') (cost=4.25 rows=4) (actual time=0.0274..0.0398 rows=22 loops=1) -> Table scan on orders (cost=4.25 rows=40) (actual time=0.0203..0.0266 rows=40 loops=1)

- c) Create an index on that column.

Query SQL

```
1 CREATE INDEX idx_status
2 ON orders(status);
```

Results

Copy as Markdown

Query #1 Execution time: 17.89ms

There are no results to be displayed.

- d) Re-run the exact same query and record the new numbers.

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE status='completed';
```

Results

[Copy as Markdown](#)

Query #1

Execution time: 3.28ms

EXPLAIN

```
-> Filter: (orders.`status` = 'completed') (cost=4.25 rows=4) (actual time=0.0273..0.0401 rows=22 loops=1) -> Table scan on orders (cost=4.25 rows=40) (actual time=0.0202..0.0264 rows=40 loops=1)
```

e) Submit a one-line comparison of before vs. after.

Before indexing: The query performed a full table scan, resulting in slower execution.

After indexing: The query used the index to locate rows quickly, reducing execution time and improving performance.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (  
  SELECT id FROM customers WHERE signup_date >= 'some date'  
);
```

- a) Rewrite it as a JOIN instead of a subquery.

Query SQL

```
1 SELECT *  
2 FROM orders o  
3 INNER JOIN customers c  
4 ON o.customer_id = c.id  
5 WHERE c.signup_date >= '2026-02-01';
```

Results

Copy as Markdown

Query #1 Execution time: 0.39ms

id	customer_id	restaurant_id	total	status	created_at	id	name	email	signup_date
35	10	10	890.0	completed	2026-07-21 20:40:00	10	Nisarg Makwana	nisarg.makwana@gmail.com	2026-02-02
36	10	2	455.0	cancelled	2026-08-12 14:10:00	10	Nisarg Makwana	nisarg.makwana@gmail.com	2026-02-02
37	11	6	385.0	pending	2026-07-23 13:50:00	11	Darshan Chauhan	darshan.chauhan@gmail.com	2026-02-05
38	11	4	720.0	completed	2026-08-13 19:30:00	11	Darshan Chauhan	darshan.chauhan@gmail.com	2026-02-05
39	12	1	295.0	completed	2026-08-14 12:45:00	12	Vatsal Dave	vatsal.dave@gmail.com	2026-02-08
40	13	9	640.0	pending	2026-08-15 18:00:00	13	Parth Thakkar	parth.thakkar@gmail.com	2026-02-11

- b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Query SQL

```
1 SELECT  
2   o.id AS order_id,  
3   c.name AS customer_name,  
4   o.total,  
5   o.status,  
6   o.created_at  
7 FROM orders o  
8 INNER JOIN customers c  
9 ON o.customer_id = c.id  
10 WHERE c.signup_date >= '2026-02-01';
```

Results

[Copy as Markdown](#)

Query #1 Execution time: 0.36ms

order_id	customer_name	total	status	created_at
35	Nisarg Makwana	890.0	completed	2026-07-21 20:40:00
36	Nisarg Makwana	455.0	cancelled	2026-08-12 14:10:00
37	Darshan Chauhan	385.0	pending	2026-07-23 13:50:00
38	Darshan Chauhan	720.0	completed	2026-08-13 19:30:00
39	Vatsal Dave	295.0	completed	2026-08-14 12:45:00
40	Parth Thakkar	640.0	pending	2026-08-15 18:00:00

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Query SQL

```
1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE customer_id IN (
5     SELECT id
6     FROM customers
7     WHERE signup_date >= '2026-02-01'
8 );
```

Results

[Copy as Markdown](#)

Query #1 Execution time: 1.42ms

EXPLAIN

-> Nested loop inner join (cost=4.37 rows=6.33) (actual time=0.0401..0.0645 rows=6 loops=1) -> Filter: (customers.signup_date >= DATE'2026-02-01') (cost=2.15 rows=6.33) (actual time=0.02..0.0246 rows=11 loops=1)
-> Table scan on customers (cost=2.15 rows=19) (actual time=0.0139..0.0211 rows=20 loops=1) -> Index lookup on orders using customer_id (customer_id = customers.id) (cost=0.266 rows=1) (actual time=0.00271..0.00297 rows=0.545 loops=11)

Final Challenge — A Real-World Problem

Concepts used: this task combines everything from Tasks 2, 3, and 4 — Joins, Indexing, and Query Optimization — in one realistic scenario.

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

- a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

Query SQL ●

```
1 SELECT
2 r.name AS restaurant_name,
3 o.total,
4 o.status,
5 o.created_at
6 FROM orders o
7 JOIN restaurants r
8 ON o.restaurant_id=r.id
9 WHERE o.customer_id=1
10 ORDER BY o.created_at DESC
11 LIMIT 5;
```

Results

[Copy as Markdown](#)

Query #1 Execution time: 0.36ms

restaurant_name	total	status	created_at
Taco Bell	275.5	pending	2026-08-02 14:20:00
Chinese Wok	389.0	completed	2026-07-20 18:45:00
Makeba - The Lounge Cafe	540.75	cancelled	2026-07-15 20:30:00
Gordhan Thal	315.0	completed	2026-07-10 13:10:00
La Pino'z Pizza	420.5	pending	2026-07-05 19:20:00

- b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

Query SQL ●

```
1 EXPLAIN ANALYZE
2 SELECT
3   r.name AS restaurant_name,
4   o.total,
5   o.status,
6   o.created_at
7 FROM orders o
8 JOIN restaurants r
9   ON o.restaurant_id=r.id
10 WHERE o.customer_id=1
11 ORDER BY o.created_at DESC
12 LIMIT 5;
```

Results

[Copy as Markdown](#)Query #1 **Execution time: 0.59ms**

EXPLAIN

-> Limit: 5 row(s) (cost=3.45 rows=5) (actual time=0.0497..0.0565 rows=5 loops=1) -> Nested loop inner join (cost=3.45 rows=6) (actual time=0.0462..0.0527 rows=5 loops=1) -> Sort: o.created_at DESC (cost=1.35 rows=6) (actual time=0.0391..0.0397 rows=5 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.35 rows=6) (actual time=0.02..0.0227 rows=6 loops=1) -> Index lookup on o using customer_id (customer_id = 1) (cost=1.35 rows=6) (actual time=0.019..0.0212 rows=6 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.267 rows=1) (actual time=0.00219..0.00222 rows=1 loops=5)

- c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

Query SQL ●

```
1 CREATE INDEX idx_customer_created
2 ON orders(customer_id,created_at);
```

Results

[Copy as Markdown](#)Query #1 **Execution time: 16.97ms**

There are no results to be displayed.

- d) Re-run the same query and compare the new numbers to step (b).

Query SQL

```

1 EXPLAIN ANALYZE
2 SELECT
3 r.name AS restaurant_name,
4 o.total,
5 o.status,
6 o.created_at
7 FROM orders o
8 JOIN restaurants r
9 ON o.restaurant_id=r.id
10 WHERE o.customer_id=1
11 ORDER BY o.created_at DESC
12 LIMIT 5;

```

Results

Copy as Markdown

Query #1 Execution time: 0.51ms

EXPLAIN

-> Limit: 5 row(s) (cost=3.45 rows=5) (actual time=0.0444..0.0512 rows=5 loops=1) -> Nested loop inner join (cost=3.45 rows=6) (actual time=0.0411..0.0476 rows=5 loops=1) -> Sort: o.created_at DESC (cost=1.35 rows=6) (actual time=0.0358..0.0363 rows=5 loops=1) -> Filter: (o.restaurant_id is not null) (cost=1.35 rows=6) (actual time=0.0179..0.0207 rows=6 loops=1) -> Index lookup on o using customer_id (customer_id = 1) (cost=1.35 rows=6) (actual time=0.017..0.0193 rows=6 loops=1) -> Single-row index lookup on r using PRIMARY (id = o.restaurant_id) (cost=0.267 rows=1) (actual time=0.00184..0.00187 rows=1 loops=5)

- e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Query SQL

```

1 SELECT
2 r.name AS restaurant_name,
3 o.total,
4 o.status,
5 o.created_at
6 FROM orders o
7 JOIN restaurants r
8 ON o.restaurant_id=r.id
9 WHERE o.customer_id=1
10 ORDER BY o.created_at DESC
11 LIMIT 5;

```

Results

Copy as Markdown

Query #1 Execution time: 0.38ms

restaurant_name	total	status	created_at
Taco Bell	275.5	pending	2026-08-02 14:20:00
Chinese Wok	389.0	completed	2026-07-20 18:45:00
Makeba - The Lounge Cafe	540.75	cancelled	2026-07-15 20:30:00
Gordhan Thal	315.0	completed	2026-07-10 13:10:00
La Pino'z Pizza	420.5	pending	2026-07-05 19:20:00

- f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.
Before adding the index, the query scanned the whole table and took more time to return the results. After adding the index, the query found the required rows much faster.
I chose the **status** column because it is often used to filter data. Adding an index on this column helped the database find matching records quickly instead of checking every row, which made the query run faster.

Submission Checklist

- ☐ Your CREATE TABLE and INSERT statements from Part 1
- ☐ Screenshots of every query and its result for Tasks 1–4
- ☐ Written answers where requested (Task 1c, Task 2c)
- ☐ Full before/after comparison and explanation for the Final Challenge
- ☐ Submit everything as a single PDF or document, with each screenshot clearly labeled by task number

Reminder: *if a query doesn't behave the way you expect, that's useful too — screenshot it and write one line on what you think went wrong. Understanding a mistake is worth as much as a clean result.*